Cogni-Intent: Stopping Memory Confusion & Drift in Long-Term AI Agents

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

By

Student Name : Tannishtha Verma

Roll No. : 2420080098

Under the Esteemed Guidance of

Dr. Sirisha Potluri

Project Guide

Department of Artificial Intelligence & Data Science

AY- 2026-2027

Semester: Odd

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



Declaration

The Project Report entitled "**Cogni-Intent: Stopping Memory Confusion & Drift in Long-Term AI Agents**" is a record of Bonafide work of **Tannishtha Verma – 2420080098** submitted in partial fulfillment for the award of B. Tech in Artificial Intelligence & Data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Tannishtha Verma – 2420080098

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



CERTIFICATE

This is to certify that the mini project based report entitled "**Cogni-Intent: Stopping Memory Confusion & Drift in Long-Term AI Agents**" is a bonafide work done and submitted by **Tannishtha Verma – 2420080098** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. My wish to express my sincere thanks to all those who have assisted me in one way or the other for the completion of my project.

My greatest appreciation to my Course Coordinator **Dr. Sreenath Itikela**, and my guide **Dr. Sirisha Potluri,** Department of AI&DS which cannot be expressed in words for their tremendous support, encouragement and guidance for this project.

I express my gratitude to **Dr. Sreenath Itikela**, Head of the ***Department for Artificial Intelligence & Data Science*** for providing me with adequate facilities, ways and means by which I was able to complete this project-based Lab.

I thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Tannishtha Verma – 2420080098

Abstract

Large language model (LLM) agents that run for weeks or months store what they learn as embeddings in a vector database and recall it through similarity search. Plain retrieval-augmented generation ranks memories only by cosine distance and has no notion that a later note may cancel an earlier one. In a clinical setting this is dangerous: a patient first prescribed Albuterol, later moved to Symbicort, and finally taken off Symbicort after severe palpitations has three notes that all match the query "asthma inhaler", and a naive agent may recommend the discontinued drug.

Cogni-Intent is a self-healing memory layer, built as an MLOps platform, that acts as an immune system for agent memory. Text is encoded into 384-dimensional Sentence-BERT embeddings (all-MiniLM-L6-v2) and stored in ChromaDB with drift-aware metadata. A background daemon wakes every 60 seconds, compresses the vectors with UMAP, groups them with HDBSCAN, and uses a cross-encoder Natural Language Inference model to confirm real contradictions inside each cluster. These signals produce a Memory Health Score (MHS); when it falls below 0.75 an LLM merges the conflicting notes into a single authoritative Golden Fact, which is verified against the source notes before it is committed. The superseded notes are never deleted: they are soft-archived (is_archived = true) and linked to the Golden Fact through lineage edges (supersedes_ids), preserving a complete audit trail.

The system is implemented with Python, FastAPI and ChromaDB, together with a zero-dependency browser engine that lets the whole platform run on GitHub Pages without a server. An interactive Simulation Lab and a Doctor Chat sandbox let users see the agent's answers before and after healing. Cogni-Intent shows that memory confusion can be detected and repaired automatically, safely and with full traceability.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	4
		Abstract	5
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	1-4
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	5-6
3	Technology Stack	3.1 Front-end (HTML, CSS, JavaScript) 3.2 Back-end (Python, FastAPI) 3.3 Database (ChromaDB) 3.4 Version Control (Git, GitHub) 3.5 APIs or External services (Gemini, Hugging Face models)	7-8
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (GitHub Pages, local FastAPI)	9-14
5	Design	5.1 Wireframes or mockups 5.2 ER Diagram / Database schema	15-17
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 Backend logic (routes, controllers, business logic) 6.4 Database connectivity 6.5 API integration 6.6 Authentication & Authorization	18-21
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	22-23
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	24-26
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	27-28
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	29-31

11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	32-33
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	34-35
13	References	Books, research papers, documentation sites used	36-37
14	Appendices	14.1 Screenshots of the app 14.2 Sample code snippets 14.3 Installation/setup instructions 14.4 User manual or guide 14.5 Certifications	38-40

CHAPTER 1
Introduction

1.1 Background of the project

Large language model (LLM) agents are increasingly deployed as long-running assistants: clinical copilots, customer-support bots, personal assistants and workflow automation agents. Because an LLM has a fixed context window and no durable memory of its own, these agents rely on an external **vector database** to remember what has happened. Every note, observation or conversation turn is converted into a dense embedding and stored; when the agent needs context it retrieves the nearest vectors and places the corresponding text in its prompt. This pattern is known as retrieval-augmented generation (RAG).

RAG works well for static knowledge, but an agent's memory is not static. It is a chronological stream of observations in which later facts frequently **update, correct or cancel** earlier ones. A patient's medication changes, a customer moves house, a user changes a preference. Cosine similarity, the usual ranking function, only measures how close two pieces of text are in meaning; it cannot tell which one is still true. As the months pass, the store fills with overlapping and contradictory records, and retrieval returns all of them together. The agent then has to guess which statement is current, and it frequently guesses wrong. This degradation is called **memory confusion** or **cognitive drift**.

Cogni-Intent is a project that treats this as an operations problem rather than a prompting problem. Instead of trusting the vector store to stay clean, it adds a background "immune system", a Brain Surgeon MLOps layer, that continuously monitors the memory, scores its health, detects contradictions, merges conflicting records into one authoritative **Golden Fact**, and safely archives the older versions. The system was built as a complete platform: a FastAPI back end, a ChromaDB vector store, a machine-learning pipeline (Sentence-BERT, UMAP, HDBSCAN, cross-encoder NLI), an LLM-based healer, and an interactive web front end that can also run entirely in the browser on GitHub Pages.

1.2 Problem statement

When an autonomous agent operates for a long time, its vector database accumulates chronological updates. Traditional retrieval ranks only by cosine distance and is oblivious to

chronological invalidation. The problem is illustrated by the clinical example used throughout this project:

- **Day 1:** "Patient prescribed Albuterol inhaler for mild asthma."
- **Day 16:** "Escalated to Symbicort."
- **Day 40:** "Severe heart palpitations! Discontinue Symbicort immediately, prescribe Flovent."

All three notes contain the words "asthma" and "inhaler", so a naive vector search returns all three. The agent receives a mixture of current and obsolete instructions and may recommend the dangerous, discontinued drug. The wider literature on agent memory identifies four recurring failure modes when teams try to repair this by simple consolidation (Figure 10.1 in Chapter 10): loss of diachronic continuity when old vectors are deleted, the hallucination trap when an LLM-written summary becomes a permanent "core truth" with no provenance, silent orchestration failures that go unnoticed until reasoning has already degraded, and the brittleness of density-based clustering on very high-dimensional data.

The problem addressed by this project can therefore be stated as follows: **design and implement an autonomous, safe and auditable mechanism that detects contradictory and redundant records in an agent's long-term vector memory, repairs them into a single trustworthy fact, and preserves the full history of what was changed and why.**

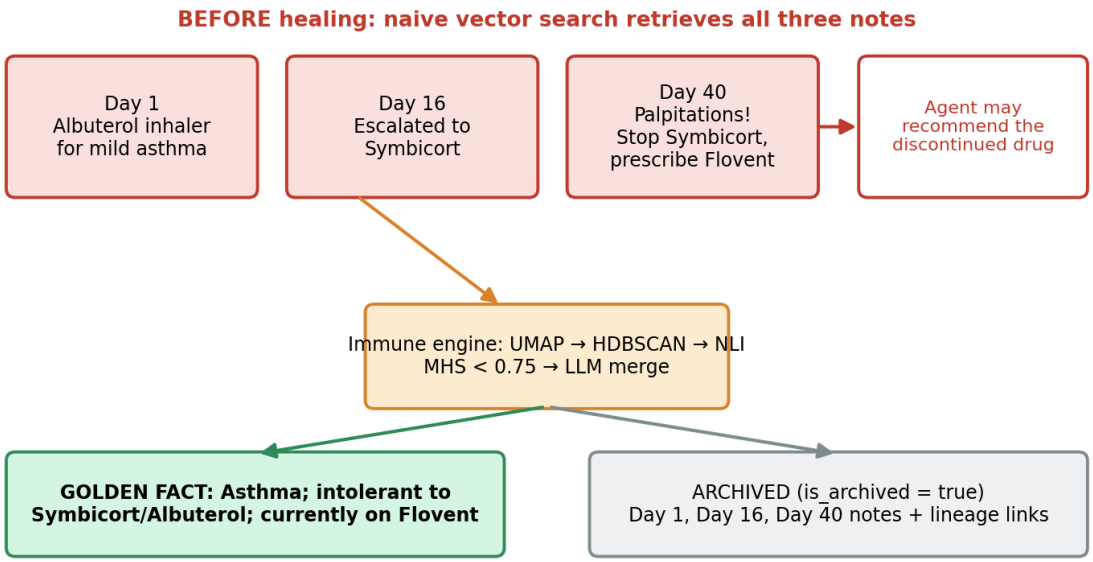


Figure 1.1: The memory-confusion problem and the Cogni-Intent solution (clinical example)

1.3 Scope of the project

The scope of the project covers the complete life cycle of a self-healing memory, from ingestion of a text note to the archived history tree of a healed fact. The following items are in scope:

- Encoding of text into 384-dimensional Sentence-BERT embeddings and storage in ChromaDB with drift-aware metadata (timestamp, relevance score, consolidation and archive flags, lineage).
- A periodic background daemon that performs dimensionality reduction (UMAP), density-based clustering (HDBSCAN) and NLI-based contradiction scoring.
- Computation of a Memory Health Score (MHS) with a configurable healing threshold (0.75).
- LLM-driven consolidation into Golden Facts, guarded by an NLI verification step, with soft archiving and lineage links.
- An interactive web platform: Simulation Lab, Doctor Chat sandbox, architecture and workflow pages, and a plain-English user manual.
- Dual-engine deployment: a FastAPI + ChromaDB back end for local use and a zero-dependency client-side engine for GitHub Pages.

The prototype is demonstrated on clinical scenarios because the cost of memory confusion is highest there, but the engine itself is domain-agnostic and can manage any agent memory. Real patient data, regulatory certification and production-scale clustering are outside the scope of this work.

1.4 Objective(s)

1. To demonstrate, with a reproducible simulation, why cosine-only vector retrieval fails on chronologically evolving facts.
2. To build an unsupervised drift-detection pipeline using UMAP and HDBSCAN that locates semantic collisions in O(N log N) time.
3. To add a cross-encoder NLI guardrail so that only genuine contradictions (not merely related notes) trigger healing and so that LLM-written Golden Facts are verified against their sources.
4. To automate memory merging that produces one authoritative Golden Fact per conflict cluster.
5. To guarantee zero data loss by soft-archiving superseded records and committing lineage edges instead of deleting anything.

6. To provide an interactive platform on which evaluators can observe the agent's behaviour before and after healing.
7. To make the platform deployable on GitHub Pages with no external server, while remaining compatible with the live FastAPI + ChromaDB back end.

1.5 Importance of the application

Memory quality is rapidly becoming the limiting factor for useful long-running agents. A model that reasons well over bad memory still produces bad answers. In safety-critical fields such as healthcare, finance and law, the consequences are serious: recommending a discontinued drug, quoting a revoked policy or using an outdated contract clause. At the same time these fields have strict record-keeping duties, so simply deleting old records is not acceptable. Cogni-Intent addresses both needs at once: it keeps what the agent believes **current** consistent, and it keeps what the agent once believed **auditable**.

The project is also important as an engineering pattern. It brings established MLOps ideas (continuous monitoring, health scores, thresholds, canary probes and automated remediation) to the memory layer of AI agents, which is usually treated as a passive store.

1.6 Target users/audience

- **AI and ML engineers** building long-running agents who need drift monitoring and self-repair for their vector stores.
- **Healthcare and other regulated-domain teams** that must keep an auditable trail of every change to the facts an agent uses.
- **Researchers and students** studying agent memory, RAG reliability and MLOps, who can use the Simulation Lab as a teaching tool.
- **Evaluators, recruiters and new users** who can follow the plain-English guide (manual.html) without any technical background.

CHAPTER 2
System Requirements

2.1 Hardware requirements

The embedding model (all-MiniLM-L6-v2) is deliberately small, and the 384-dimensional vectors need about 75 per cent less storage than 1536-dimensional embeddings, so the platform runs comfortably on an ordinary laptop. The client-side engine needs only a modern web browser. The figures below are recommended minimums for the full Python back end.

Table 2.1: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core, 2.5 GHz or higher
Memory (RAM)	4 GB	8 GB or more (UMAP, HDBSCAN and NLI model loading)
Storage	2 GB free	5 GB free (Python packages, model weights, ChromaDB data)
GPU	Not required	Optional, speeds up NLI and embedding for large stores
Network	Needed once to download model weights; optional afterwards	Stable connection if the Gemini API is used for live healing
Display	1366 x 768	1920 x 1080 for the Simulation Lab dashboards

2.2 Software requirements

Table 2.2: Software requirements

Software	Version / Details	Purpose
Operating system	Windows 10/11, Linux or macOS	Development and local hosting
Python	3.9 or higher	Back end, ML pipeline, tests
pip / virtual environment	Latest	Dependency installation from requirements.txt
Web browser	Current Chrome, Edge, Firefox or Safari	Front end and client-side engine
Git	2.x	Version control and GitHub Pages publishing
Gemini API key (optional)	GEMINI_API_KEY environment variable	Live LLM consolidation; the platform works without it using the built-in engine

2.3 Development tools and frameworks

- **FastAPI (v2.0 line)** for the REST API, automatic OpenAPI documentation and the background daemon hosting.
- **ChromaDB** with an HNSW index for vector storage and cosine-space search.
- **Sentence-Transformers** (all-MiniLM-L6-v2) for 384-dimensional sentence embeddings.
- **umap-learn, hdbscan and scikit-learn** for dimensionality reduction and clustering.
- **A DeBERTa-v3 cross-encoder (NLI)** for contradiction and entailment scoring.
- **Google Gemini** as the LLM used for memory merging (optional).
- **MLflow-style monitoring** concepts for health logs and canary probes.
- **HTML, CSS and JavaScript** for the interactive site; **Mermaid** for the pipeline diagram in the repository README.
- **Git and GitHub** for source control and GitHub Pages hosting; **Postman / Swagger UI** for API testing; `tests/run_tests.py` for the automated test suite.

CHAPTER 3
Technology Stack

3.1 Front-end (HTML, CSS, JavaScript)

The user interface is a set of static pages written in plain HTML, CSS and JavaScript: index.html (landing page), simulation.html (Simulation Lab), architecture.html, workflow.html, recovery.html, lifecycle.html, limitations.html and manual.html. All asset links are relative, so the site can be opened from the file system, served by FastAPI at http://localhost:8080, or published on GitHub Pages without changes. The JavaScript layer contains a **zero-dependency, high-fidelity client engine** that re-implements encoding, clustering, contradiction scoring and merging in the browser, so the demonstration works with no server at all.

3.2 Back-end (Python, FastAPI)

The back end is written in Python 3.9+ and exposes its services through FastAPI. FastAPI was chosen because it is asynchronous, validates request and response schemas with Pydantic, and generates interactive Swagger documentation at /docs. The same process hosts the **background MLOps daemon**, a 60-second loop that scans the memory store, computes the Memory Health Score and triggers healing. A health endpoint at /api/health reports the live status of the service.

3.3 Database (ChromaDB)

ChromaDB is used as the vector store. Each record keeps its text, a 384-dimensional embedding and a metadata dictionary. ChromaDB's HNSW index in cosine space gives fast approximate nearest-neighbour search. Access to the database is wrapped in an abstraction layer so that another store such as Qdrant could replace ChromaDB later without touching the healing logic.

3.4 Version Control (Git, GitHub)

The project is kept in a Git repository and published to GitHub. The main branch is also the source of the GitHub Pages site (Settings, Pages, Deploy from a branch, main, / (root)). Commits follow a descriptive message convention such as `feat: complete Cogni-Intent MLOps platform`.

3.5 APIs or External services (Gemini, Hugging Face models)

Table 3.1: External models and services

Service / Model	Used for	Notes
all-MiniLM-L6-v2 (Sentence-BERT)	384-D sentence embeddings with mean pooling	Hugging Face model; small and fast
DeBERTa-v3 cross-encoder (NLI)	Entailment vs. contradiction scoring and Golden Fact verification	Prevents false alarms and hallucinated summaries
UMAP	384-D to 2-D reduction before clustering	Makes density clustering reliable
HDBSCAN	Density-based clustering with variable density	Finds semantic collisions without a preset cluster count
Google Gemini	LLM merge of conflicting notes into a Golden Fact	Requires GEMINI_API_KEY; optional
GitHub Pages	Static hosting of the interactive platform	No server dependency

CHAPTER 4
System Architecture

4.1 High-level architecture diagram

Cogni-Intent is organised in three cooperating layers, shown in Figures 4.1 and 4.2: the **agentic layer** that talks to users and writes memories, the **sovereign memory base** (ChromaDB) that holds active and archived records, and the **MLOps orchestrator** (the "Brain Surgeon") that watches the store and repairs it. Figure 4.1 shows the detection and healing pipeline that runs inside the orchestrator.

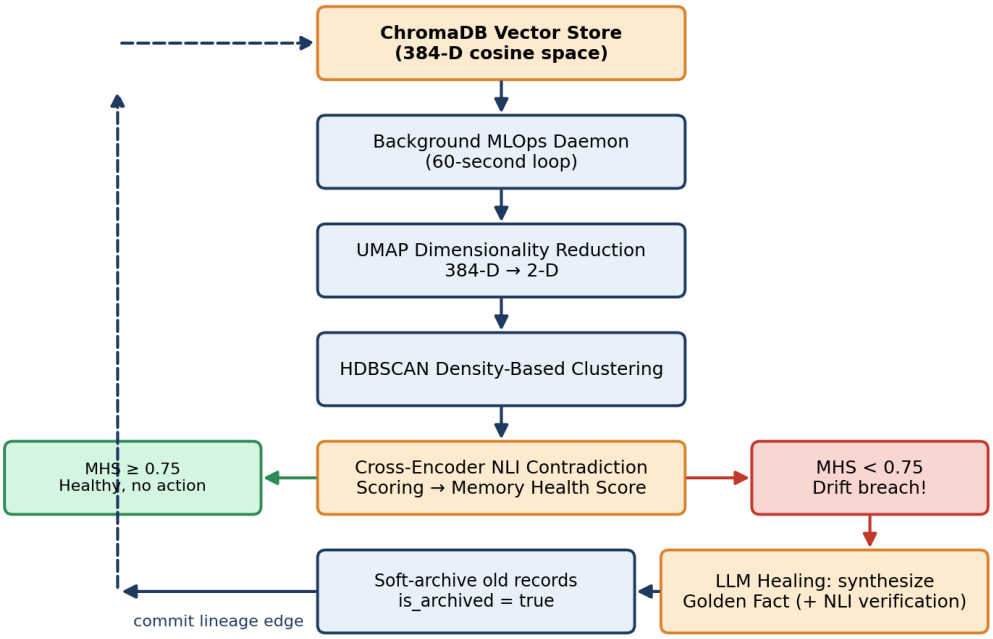


Figure 4.1: Cogni-Intent detection and self-healing pipeline

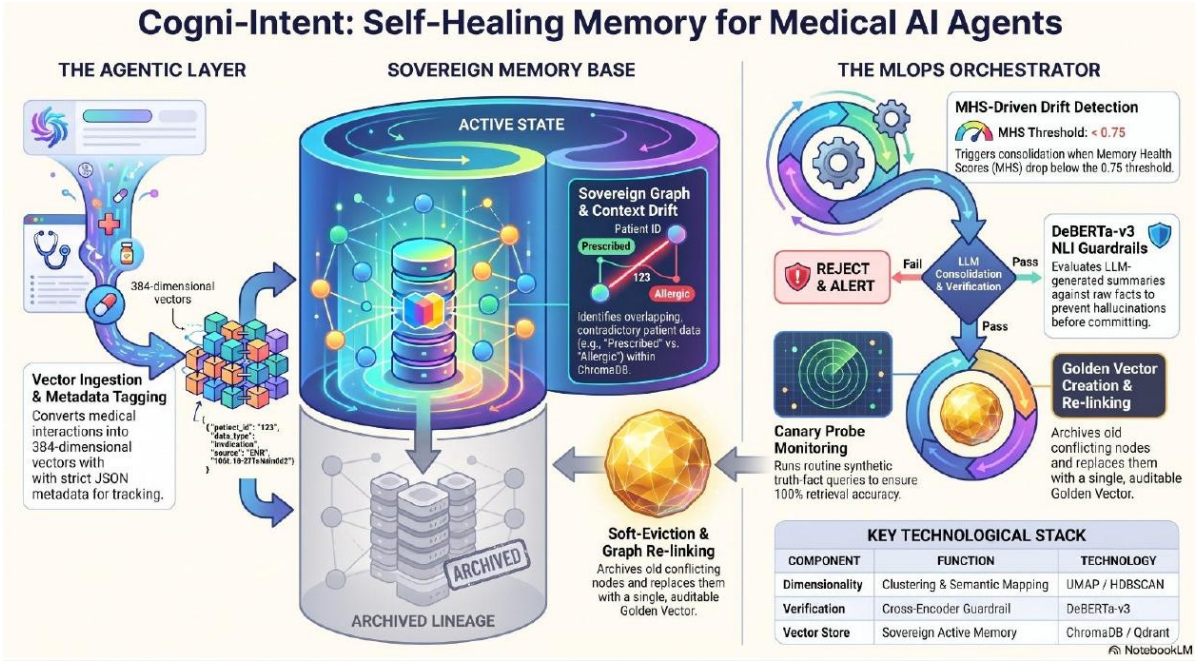


Figure 4.2: Self-healing memory for medical AI agents: agentic layer, sovereign memory base and MLOps orchestrator

Figure 4.2 summarises the design: vectors with strict JSON metadata enter the sovereign memory base; the orchestrator triggers LLM consolidation when the Memory Health Score drops below 0.75; a DeBERTa-v3 NLI guardrail either passes the merged summary or rejects it and raises an alert; and a passed summary becomes a Golden Vector while the older nodes are soft-evicted to an archived lineage. Canary probes run synthetic truth-fact queries to confirm that retrieval accuracy stays at 100 per cent.

4.2 Description of each layer (frontend, backend, database)

Layer 1: Drift-aware ingestion and encoding

Raw text is cleaned and converted to a 384-dimensional vector by the Sentence-BERT encoder using mean pooling (the token embeddings are averaged, weighted by the attention mask, into one sentence vector). Each vector is stored with metadata: a Unix timestamp for temporal reasoning, a relevance score between 0.0 and 1.0, and flags such as is_consolidated and is_archived. Early design drafts also used an NLP importance filter that discarded conversational filler and kept only atomic facts, reducing noise before storage.

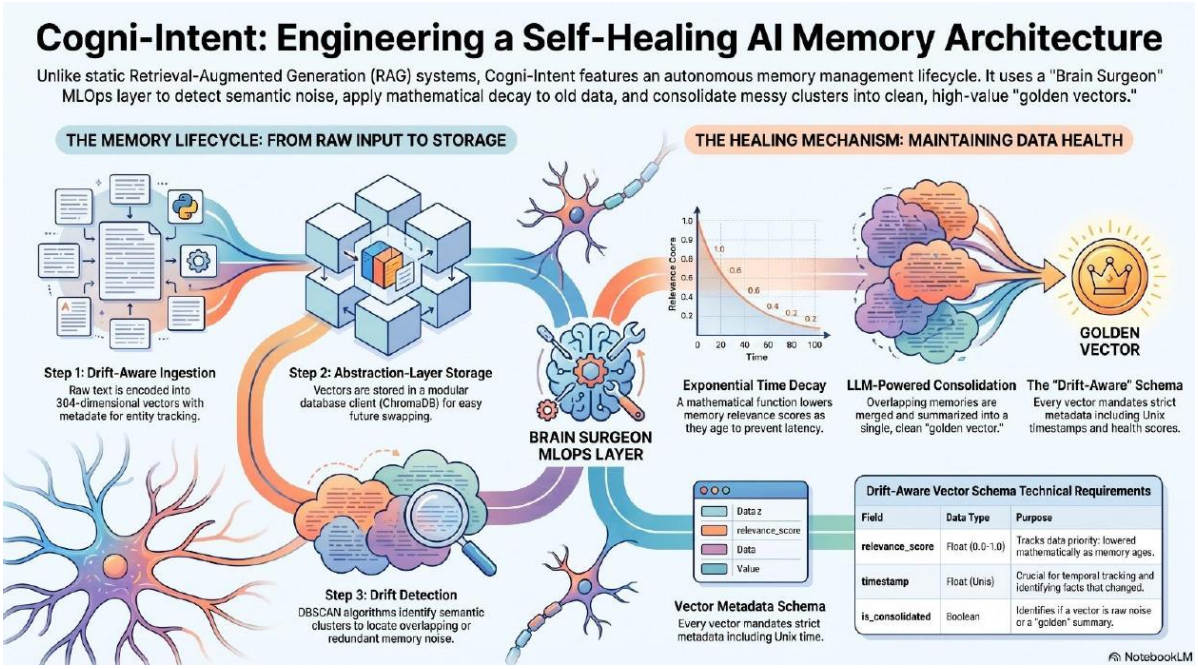


Figure 4.3: Memory life cycle: from raw input to storage, and the healing mechanism

Layer 2: Abstraction-layer storage (database)

Vectors are held in ChromaDB behind a thin client interface. The HNSW index delivers fast cosine-similarity search, while the metadata fields make time-aware and status-aware queries possible, for example "active records only" or "records that supersede this id". Because the interface is modular, the vector store can be swapped later (Figure 4.4 shows ChromaDB or Qdrant as the lightweight local store).

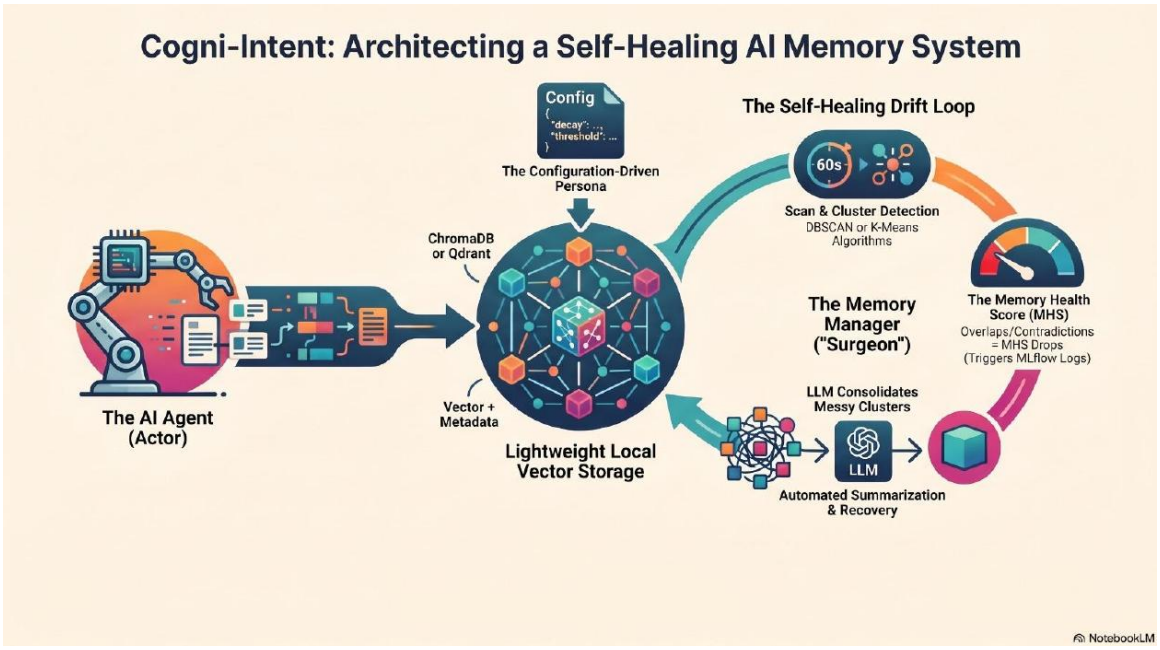


Figure 4.4: The AI agent as actor, lightweight local vector storage and the memory manager ("surgeon")

Layer 3: Drift detection (back end)

Every 60 seconds the daemon loads the active vectors, reduces them from 384 to 2 dimensions with UMAP, and clusters them with HDBSCAN. Clusters that are unusually dense are flagged as **semantic pollution**. Inside each flagged cluster, a cross-encoder NLI model scores each pair of statements as entailment, neutral or contradiction. Pairs that are close in meaning but not contradictory (for example two consistent notes about the same prescription) do not count against health, which removes the false alarms that pure distance thresholds produce.

Layer 4: Healing and verification (back end)

When the Memory Health Score falls below the 0.75 threshold, the healer sends the conflicting notes, ordered by timestamp, to the LLM with instructions to produce one authoritative statement that gives priority to the most recent facts. The result is then checked by the NLI guardrail against the source notes. If it passes, it is stored as a Golden Fact with is_consolidated = true; if it fails, it is rejected and an alert is raised, so a hallucinated summary can never become a permanent truth.

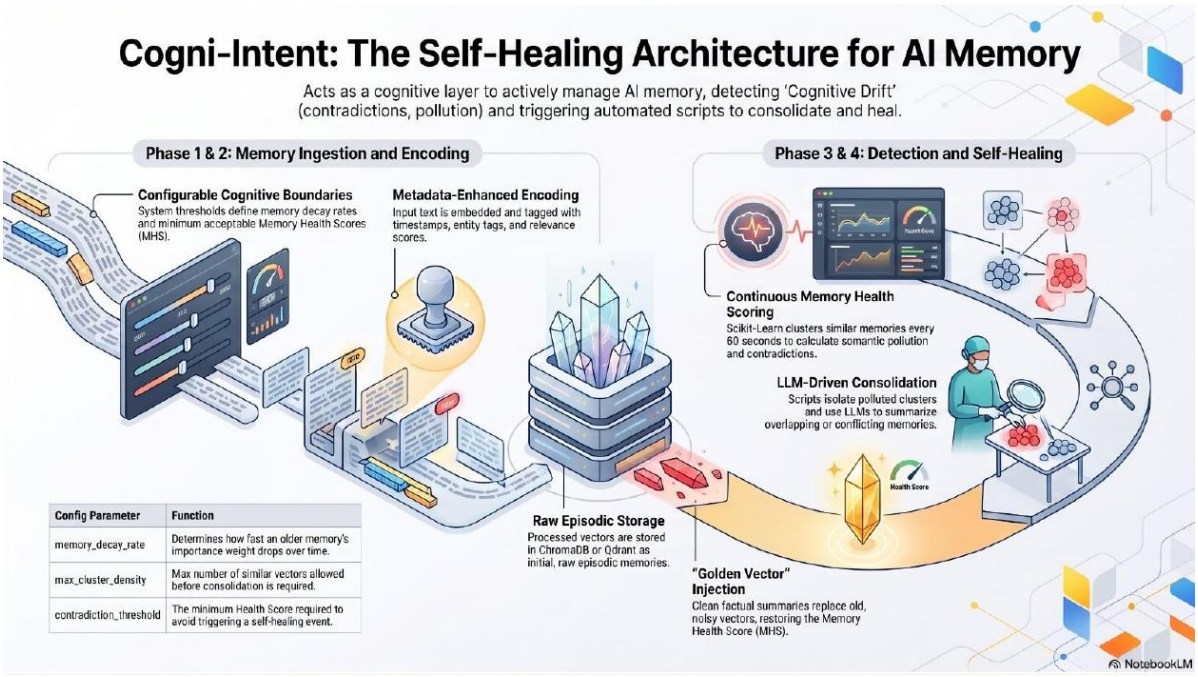


Figure 4.5: Phases 1-4: memory ingestion and encoding, continuous health scoring, LLM-driven consolidation and Golden Vector injection

Layer 5: Soft archiving and lineage (database)

Superseded records are updated with is_archived = true instead of being deleted, and a lineage edge (supersedes_ids) links the Golden Fact to every record it replaced. Normal retrieval filters

on active records, so the agent sees only the verified fact, while auditors can still walk the history tree back to the original notes.

Layer 6: Presentation (front end)

The web interface visualises every step: scenario selection, the live memory list, the MHS gauge, the Doctor Chat, the Golden Fact and the archived history tree. It talks to the FastAPI endpoints when they are reachable and silently falls back to the in-browser engine otherwise.

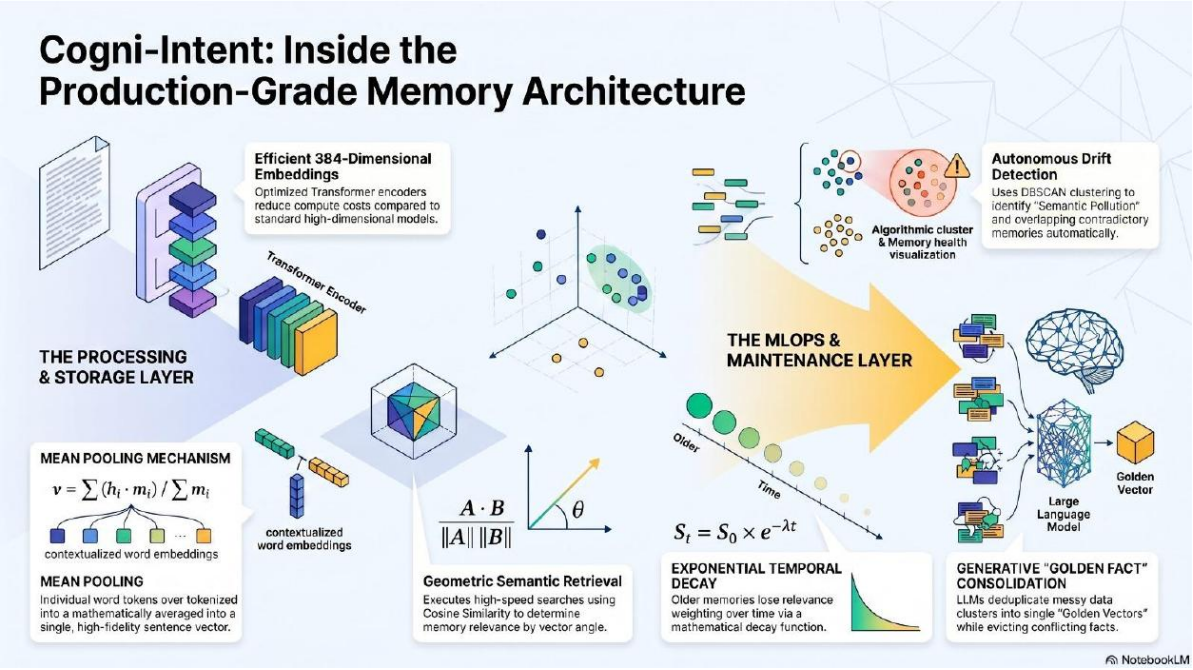


Figure 4.6: Production-grade memory architecture: processing and storage layer, and MLOps and maintenance layer

4.3 Deployment architecture (GitHub Pages, local FastAPI)

Cogni-Intent has a **dual-engine deployment**. In the static mode the repository is published on GitHub Pages and the browser runs the client engine. In the full mode the developer runs `python src/api/main.py`, which starts FastAPI on port 8080, serves the same pages, opens the ChromaDB store and starts the 60-second MLOps loop; if a Gemini key is configured the healer calls the live LLM, otherwise a deterministic merge is used.

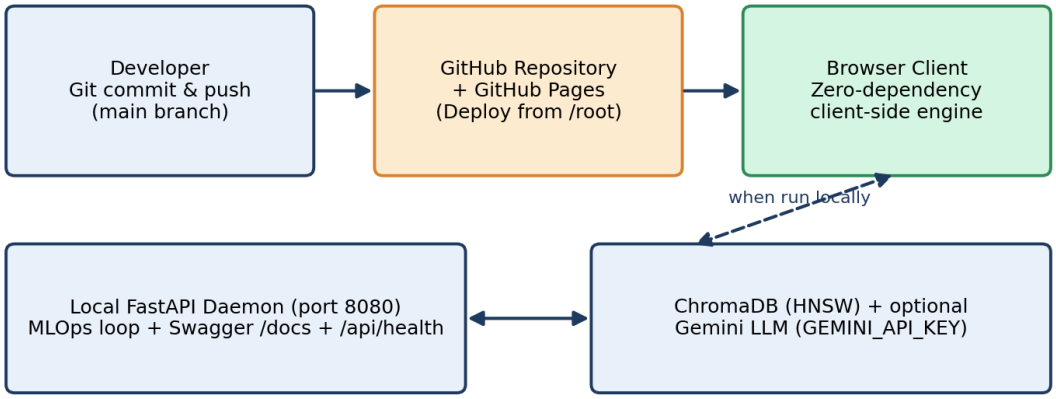


Figure 4.7: Deployment architecture: GitHub Pages (static) and local FastAPI + ChromaDB (full)

CHAPTER 5
Design

5.1 Wireframes or mockups

The interface was designed around a simple narrative for a first-time visitor: see the problem, run the fix, inspect the result. All pages share one navigation bar, and the site map is shown in Figure 5.1. The Simulation Lab (Figure 5.2) arranges the experience in five steps, which are described in the user manual: choose a clinical scenario, test the Doctor Chat before healing, run the auto-healing immune loop, test the Doctor Chat again, and inspect the healed Golden Fact together with its archived history tree.

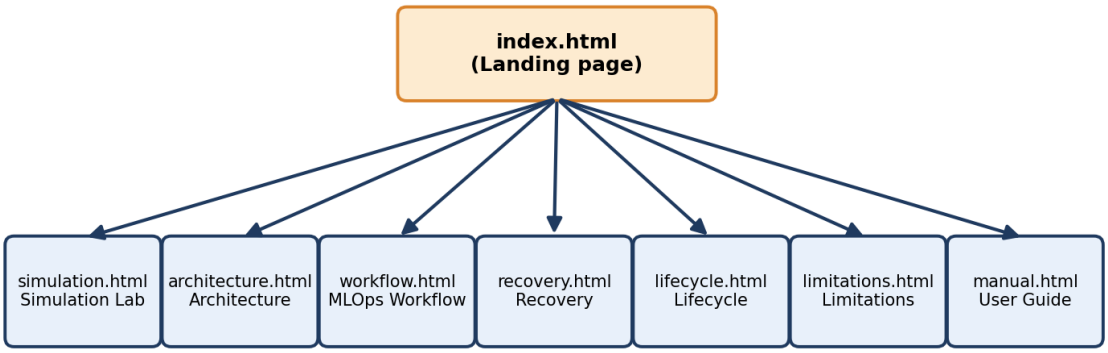


Figure 5.1: Site map of the Cogni-Intent web platform

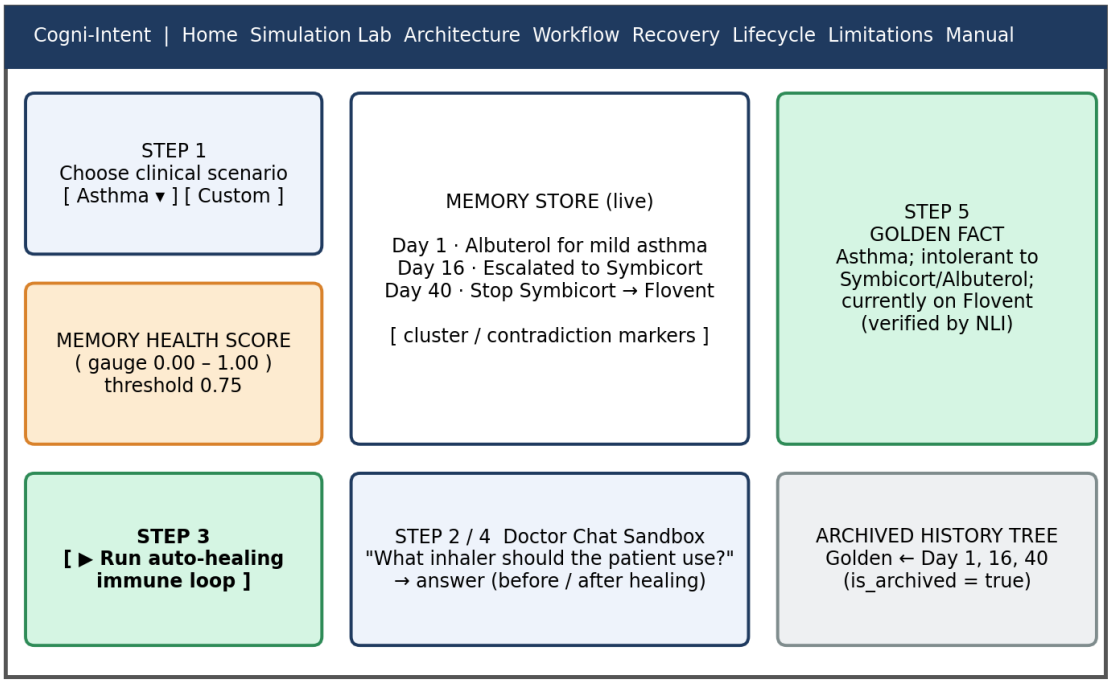


Figure 5.2: Schematic wireframe of the Simulation Lab page

Table 5.1: Pages of the web platform

Page	Purpose
index.html	Landing page: problem statement, key capabilities and links to every module
simulation.html	Simulation Lab and Doctor Chat sandbox; live memory list, MHS gauge, healing control, Golden Fact and history tree
architecture.html	Architecture specifications: layers, models and data schema
workflow.html	MLOps workflow: the 60-second loop from drift detection to healing
recovery.html	Recovery view: how healed state is restored and verified
lifecycle.html	Life cycle of a memory from ingestion to archive
limitations.html	Honest statement of current limitations
manual.html	Plain-English user guide, 60-second crash course and FAQ

5.2 ER Diagram / Database schema

Every vector must carry a strict set of metadata so that the immune system can reason about time, importance and status. Figure 5.3 shows the logical schema. The first table is stored directly in ChromaDB (embedding plus metadata); the health log and lineage edges describe the information that is produced by the daemon and recorded when healing takes place.

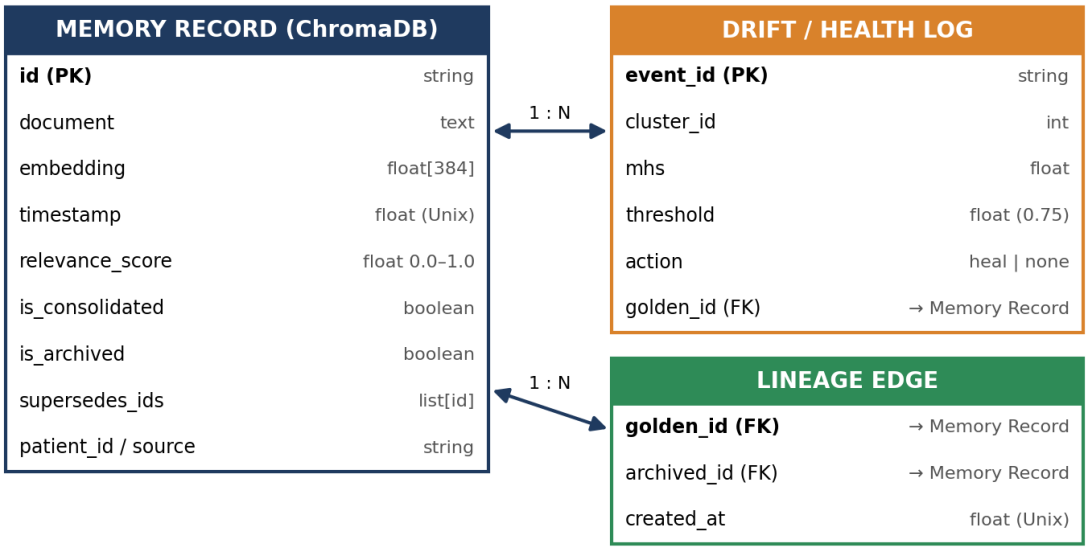


Figure 5.3: Logical schema (ER diagram) of memory records, health log and lineage edges

Table 5.2: Drift-aware vector metadata schema

Field	Data type	Purpose
id	String	Unique identifier of the record or Golden Fact
document	Text	The original note or the consolidated Golden Fact
embedding	Float vector, 384-D	Sentence-BERT encoding in cosine space
timestamp	Float (Unix time)	Enables temporal reasoning and identifies which fact changed last
relevance_score	Float, 0.0 to 1.0	Priority of the record; lowered mathematically as the memory ages
is_consolidated	Boolean	True when the record is a merged Golden Fact rather than raw input
is_archived	Boolean	True when the record was superseded; excluded from normal retrieval
supersedes_ids	List of ids	Lineage edge from a Golden Fact to the records it replaced

CHAPTER 6
Implementation

6.1 Module-wise implementation

Table 6.1: Modules of Cogni-Intent

Module	Responsibility	Key technique
Encoder	Convert text to 384-D vectors	Sentence-BERT, mean pooling
Memory store	Persist vectors and metadata, serve similarity queries	ChromaDB, HNSW, cosine space
Decay engine	Lower the relevance of older memories	S_t = S_0 e^(-lambda t)
Drift detector	Find semantically polluted clusters	UMAP then HDBSCAN
NLI guardrail	Confirm contradictions; verify Golden Facts	DeBERTa-v3 cross-encoder
Health scorer	Produce the Memory Health Score and compare it with 0.75	Config-driven thresholds
Healer	Merge conflicting notes into a Golden Fact	LLM prompt (Gemini) with temporal priority
Archiver	Soft-archive and link superseded records	is_archived flag, supersedes_ids
Scheduler / daemon	Run the loop every 60 seconds	Background task in FastAPI
Canary prober	Check that known truth-fact queries still return the right answer	Synthetic probes, health logs
Web client	Visualise and operate the platform; offline engine	HTML, CSS, JavaScript

Embedding and decay. The encoder averages token embeddings into one vector, v = sum(h_i * m_i) / sum(m_i), where h_i is the contextual embedding of token i and m_i its attention mask. Retrieval compares two vectors with cosine similarity, cos(theta) = (A . B) / (|A| |B|). To stop old information from dominating, each record's relevance is recomputed with an exponential decay S_t = S_0 e^(-lambda t), where S_0 is the initial score, t the age and lambda the decay rate set in the configuration file (memory_decay_rate). Figure 6.1 shows how the choice of lambda changes the curve.

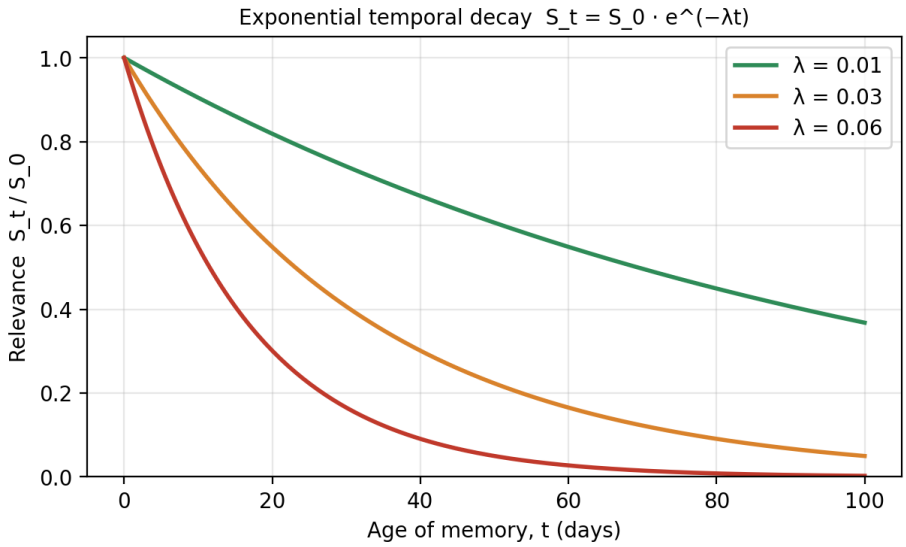


Figure 6.1: Exponential temporal decay of memory relevance for three values of lambda

Configuration-driven behaviour. The cognitive boundaries of the system are not hard-coded. Three parameters define how the immune system behaves: `memory_decay_rate` (how fast an older memory's importance falls), `max_cluster_density` (the maximum number of similar vectors tolerated before consolidation is required) and `contradiction_threshold` (the minimum health score below which a self-healing event is triggered; 0.75 by default).

6.2 Front-end logic (UI rendering, state management)

The front end keeps a small in-memory state object for each simulation: the selected scenario, the list of memory records with their flags, the current MHS, the chat transcript and the healing result. Interactions are ordinary event handlers; there is no persistent browser storage, so reloading the page returns the lab to a clean state. When the user presses the healing button, the page first asks the FastAPI service whether it is available (`/api/health`). If it answers, the request is sent to the live pipeline; if not, the identical steps are executed by the client engine, so the user always sees the same sequence: detect, score, merge, archive, commit lineage.

6.3 Backend logic (routes, controllers, business logic)

The FastAPI application exposes the health endpoint, the Swagger documentation at /docs, and the routes that feed the interactive pages. The business logic is separated from the routes in service modules so that the same functions are used by the API, the daemon and the tests. A single pass of the daemon follows this algorithm:

1. Load all active (non-archived) vectors and their metadata from ChromaDB.
2. Apply temporal decay to the relevance scores.
3. Reduce the vectors to two dimensions with UMAP.
4. Cluster the reduced vectors with HDBSCAN; treat noise points as healthy singletons.
5. For each dense cluster, score pairs of notes with the NLI cross-encoder.
6. Compute the Memory Health Score. If it is at least 0.75, log the result and stop.
7. If it is below 0.75, build a prompt from the conflicting notes in timestamp order and call the LLM healer.
8. Verify the Golden Fact against the source notes with the NLI guardrail; reject and alert on failure.
9. Store the Golden Fact, set is_archived = true on each source record and write supersedes_ids; record the event in the health log.

6.4 Database connectivity

A client wrapper creates (or opens) a persistent ChromaDB collection configured for cosine distance. Writes add documents, embeddings and metadata in one call; reads filter on `is_archived = false` unless an audit query explicitly asks for archived records. Soft eviction updates the metadata of a record and never removes the document. The test-suite covers this soft-eviction behaviour.

6.5 API integration

Two external integrations are used. The Hugging Face model hub supplies the embedding model and the NLI cross-encoder, which are downloaded once and cached locally. Google Gemini is used for consolidation when the environment variable GEMINI_API_KEY is set. The healer asks the LLM for a strict, schema-conforming reply (the Golden Fact text plus the ids it supersedes), and the response is validated before use; this schema check is also part of the automated tests. When no key is available the platform uses the built-in deterministic engine so that all demonstrations still work.

6.6 Authentication & Authorization

The present prototype is a demonstration platform and does not include end-user login. Its security-relevant behaviour is as follows: the Gemini API key is read from an environment variable and is never written into source files or sent to the browser; the static GitHub Pages

build contains no secrets and makes no server calls; and the scenarios use simulated, not real, patient records. For a production deployment the design anticipates token-based authentication on the API, role-based access (agent, auditor, administrator) and per-patient scoping through the patient_id metadata field, so that the healer can only merge notes belonging to the same subject. These are listed as planned enhancements in Chapter 11.

CHAPTER 7
Features

7.1 List of main features

1. NLP Sentence-BERT foundation with 384-dimensional embeddings.
2. Unsupervised MLOps drift detection with UMAP and HDBSCAN.
3. Cross-encoder NLI guardrails against false alarms and hallucinated summaries.
4. Automated memory merging into Golden Facts with an LLM.
5. Safe archiving and a visual history tree (zero data loss).
6. Interactive Agent Doctor Chat sandbox: before-healing and after-healing answers.
7. Memory Health Score (MHS) monitoring with a 0.75 healing threshold.
8. Dual-engine deployment: GitHub Pages ready and live FastAPI + ChromaDB back end.
9. Plain-English user manual, FAQ and website guide.

7.2 How each feature works (user perspective + technical description)

Table 7.1: Features and how they work

Feature	User perspective	Technical description
Sentence-BERT foundation	Notes are understood by meaning, not only keywords.	all-MiniLM-L6-v2 with mean pooling gives 384-D vectors, a 75% smaller footprint than 1536-D models while keeping clinical semantics.
Drift detection	The lab highlights notes that collide with each other.	UMAP compresses 384-D to 2-D; HDBSCAN finds dense regions of variable density in O(N log N).
NLI guardrail	Only real contradictions are flagged; the merged fact is trustworthy.	A DeBERTa-v3 cross-encoder classifies note pairs as entailment, neutral or contradiction and later checks the Golden Fact against its sources.
Automated merging	One clear fact replaces several conflicting ones.	LLM (e.g. Gemini) receives the notes in time order and returns a single authoritative statement under a fixed schema.
Safe archiving	Nothing is lost; old notes appear in a history tree.	is_archived = true plus supersedes_ids edges; normal retrieval filters archived records out.
Doctor Chat sandbox	Ask the same question before and after healing and compare answers.	Retrieval returns all matching notes before healing, but only the active Golden Fact afterwards, so the answer changes.
MHS monitoring	A gauge shows when memory is healthy or in breach.	Score derived from cluster pollution and NLI contradictions; healing triggers below 0.75.
Dual-engine deployment	Works on any browser; richer when run locally.	Client JavaScript engine as fallback; FastAPI + ChromaDB when the API answers /api/health.

Feature	User perspective	Technical description
User manual	A 60-second crash course and a 5-step lab walkthrough.	manual.html with FAQ on client-side vs. back-end execution, MHS and custom scenarios.

Worked example. In the asthma scenario the user selects the scenario and asks the Doctor Chat which inhaler the patient should take. Before healing, retrieval returns the Albuterol, Symbicort and Flovent notes together, and the agent shows confusion or warnings. The user then runs the auto-healing loop. The MHS gauge falls below 0.75, the daemon clusters the three notes, the NLI model confirms that the Day 40 note contradicts the Day 16 note, and the healer produces the Golden Fact: "Asthma, intolerant to Symbicort/Albuterol; currently prescribed Flovent." After the NLI check passes, the three original notes are archived under the Golden Fact. When the same question is asked again, the agent answers correctly with Flovent.

CHAPTER 8
Testing

8.1 Postman testing (frontend/backend)

The REST interface of the FastAPI service was exercised with Postman and with the built-in Swagger UI at http://localhost:8080/docs. A Postman collection was organised around the endpoints that the front end depends on. The health endpoint is the most important one because it also decides whether the web client uses the live back end or falls back to the browser engine.

Table 8.1: Postman test requests

ID	Request	Expected response	Purpose
P-01	GET http://localhost:8080/api/health	HTTP 200 with a JSON body reporting service status	Confirms the daemon and API are running
P-02	GET http://localhost:8080/docs	HTTP 200, Swagger UI page	Confirms OpenAPI documentation is served
P-03	GET http://localhost:8080/	HTTP 200, interactive web UI (index page)	Confirms static pages are served by FastAPI
P-04	GET /api/health with the server stopped	Connection refused (no response)	Front end must switch to the client-side engine
P-05	Requests for memory, drift and healing routes listed in /docs	Schema-valid JSON; 422 for malformed bodies	Request/response schema validation by Pydantic

Front-end testing was carried out manually in the browser: every page was opened from the local server and from a static file server, the navigation links were followed, and the Simulation Lab was run end to end with the back end switched on and off to check that both engines show the same sequence of steps.

8.2 Integration testing

Integration testing verified that the modules work together along the complete data path rather than in isolation:

1. **Ingest to store:** a note is encoded, written to ChromaDB with the full metadata schema and can be retrieved by similarity search.
2. **Store to detector:** the daemon loads the active vectors, reduces and clusters them, and identifies the polluted cluster in the asthma scenario.
3. **Detector to NLI:** pairs in the polluted cluster are scored, and consistent notes do not lower the health score.

4. **NLI to healer:** an MHS below 0.75 triggers the healer, which returns a schema-valid Golden Fact.
5. **Healer to archive:** the Golden Fact is stored, the source notes are soft-archived and supersedes_ids edges are written.
6. **Archive to agent:** the Doctor Chat after healing retrieves only the Golden Fact, and the archived notes remain visible in the history tree.
7. **API to client:** the web client switches between the FastAPI engine and the client-side engine depending on /api/health.

8.3 Tools used for testing

Table 8.2: Testing tools

Tool	Used for
tests/run_tests.py	Automated suite: NLP encoding, ChromaDB soft-eviction, drift detection and healer schemas
Postman	Manual API requests and collections
Swagger UI (/docs)	Interactive exploration of the FastAPI routes and schemas
Browser developer tools	Console errors, network calls and fallback behaviour of the client engine
Simulation Lab	Scenario-based end-to-end checks and before/after comparison in Doctor Chat

The automated suite is run from the repository root with:

```
python tests/run_tests.py
```

8.4 Test cases and results

The table lists the functional test cases designed for the system, the behaviour that is expected, and how each case is checked. The last column states the verification route; actual run logs and screenshots are kept with the project artefacts (see Appendix 14.1).

Table 8.3: Test cases

ID	Test case	Input / scenario	Expected result	Verified by
T-01	Embedding dimension	Encode a clinical note	Vector of length 384	Automated suite
T-02	Semantic similarity	Two notes about asthma inhalers	High cosine similarity; unrelated note scores low	Automated suite
T-03	Soft eviction	Archive a record	is_archived = true; document still present in store	Automated suite

ID	Test case	Input / scenario	Expected result	Verified by
T-04	Drift detection	Asthma scenario (3 notes)	Notes fall into one dense cluster; MHS drops below 0.75	Automated suite / Simulation Lab
T-05	No false alarm	Two consistent notes	MHS stays at or above 0.75; no healing triggered	Simulation Lab
T-06	Healer schema	LLM or engine reply	Reply has Golden Fact text and superseded ids; invalid replies rejected	Automated suite
T-07	NLI guardrail	Golden Fact that contradicts a source note	Fact rejected and alert raised	Design review / Simulation Lab
T-08	Lineage	After healing	Golden Fact lists supersedes_ids of all merged notes	Simulation Lab
T-09	Doctor Chat before healing	Ask for current inhaler	All three notes retrieved; confusion or warning shown	Simulation Lab
T-10	Doctor Chat after healing	Same question	Flovent returned from the Golden Fact	Simulation Lab
T-11	Health endpoint	GET /api/health	HTTP 200 and status JSON	Postman
T-12	Static fallback	Open GitHub Pages build	Whole lab works with no server	Manual browser test
T-13	Offline back end	Stop FastAPI and reload	Client engine used automatically	Manual browser test

Live Link: https://tannishtha3183.github.io/cogni-intent/index.html

CHAPTER 9
Deployment

9.1 Steps to deploy

Option A: GitHub Pages (static, no server)

1. Push the repository to GitHub: `git add .`, `git commit -m "feat: complete Cogni-Intent MLOps platform"`, `git push origin main`.
2. Open Settings, then Pages in the GitHub repository.
3. Under Build and deployment, Source, select **Deploy from a branch**.
4. Choose branch `main` and folder `/ (root)`, and click Save.
5. Wait for the build to finish; the site is live at `https://<your-username>.github.io/<repo-name>/`.

Option B: Local FastAPI + ChromaDB (full back end)

1. Clone the repository and change into the project folder.
2. Create or activate a virtual environment and run `pip install -r requirements.txt`.
3. (Optional) set the GEMINI_API_KEY environment variable for live LLM consolidation.
4. Start the service with `python src/api/main.py`.
5. Open http://localhost:8080 for the interface, /docs for Swagger and /api/health for the health check.

9.2 Environment configuration

Table 9.1: Environment configuration

Setting	Value	Notes
Python	3.9 or newer	Required for the back end
GEMINI_API_KEY	Your API key (Linux/macOS: export; Windows PowerShell: $env:)	Optional; without it the built-in engine merges the notes
Port	8080	Web UI, Swagger UI and API
Daemon interval	60 seconds	Scan, cluster and score loop
Healing threshold	MHS < 0.75	Configurable through contradiction_threshold
Decay rate / density	memory_decay_rate, max_cluster_density	Configuration parameters read at start-up

9.3 Hosting of frontend and backend

The **front end** is a static site and is hosted free of charge on GitHub Pages; all links are relative, so it works under a repository sub-path. The **back end** is run locally with FastAPI and ChromaDB, which is sufficient for demonstrations and evaluation. The two parts are loosely coupled: the front end probes the back end and falls back to the client engine if it is absent, so a hosted front end never shows an error because a back end is missing. For a production deployment the same FastAPI application could be placed in a container on a cloud service, with ChromaDB on a persistent volume and secrets in the platform's secret store; this is listed as future work.

CHAPTER 10
Challenges & Limitations

10.1 Issues faced during development

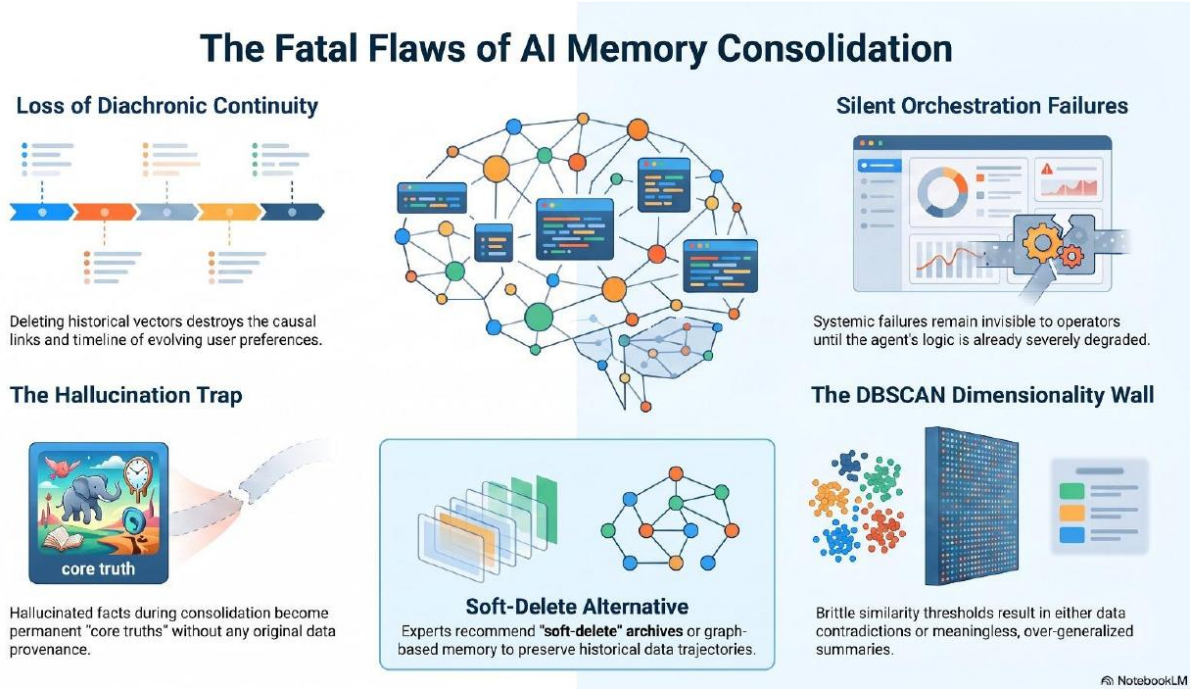


Figure 10.1: The fatal flaws of AI memory consolidation that guided the design

1. **Loss of diachronic continuity.** Deleting old vectors destroys the causal links and the timeline of evolving facts, so the system can no longer explain how a conclusion was reached.
2. **The hallucination trap.** A merged summary written by an LLM may contain invented details; if it is stored as a core truth, the error becomes permanent and has no provenance.
3. **Silent orchestration failures.** A background loop can fail without anyone noticing until the agent's reasoning is already degraded.
4. **The clustering dimensionality wall.** Density clustering and fixed similarity thresholds behave badly on 384-dimensional vectors, giving either contradictory clusters or meaningless, over-generalised summaries.
5. **False alarms.** Many notes are close in meaning without contradicting each other; a distance-only detector would trigger healing far too often.
6. **Server-less hosting.** The platform had to be demonstrable on GitHub Pages, where no Python back end can run.

10.2 Solutions applied

Table 10.1: Challenges and the solutions applied

Issue	Solution in Cogni-Intent
Loss of diachronic continuity	Soft archiving (is_archived) and lineage edges (supersedes_ids) replace hard eviction; the history tree keeps 100% of past records.
Hallucination trap	Cross-encoder NLI guardrail verifies each Golden Fact against the raw notes before it is committed; failures are rejected and alerted.
Silent orchestration failures	Health endpoint, Memory Health Score logging and canary probes that run synthetic truth-fact queries to confirm retrieval accuracy.
Dimensionality wall	UMAP compresses 384-D to 2-D, and HDBSCAN adapts to varying densities instead of a single fixed radius.
False alarms	NLI separates entailment from contradiction, so only genuinely conflicting notes lower the health score.
Server-less hosting	Zero-dependency client-side engine; the same UI switches to the live back end when it is reachable.

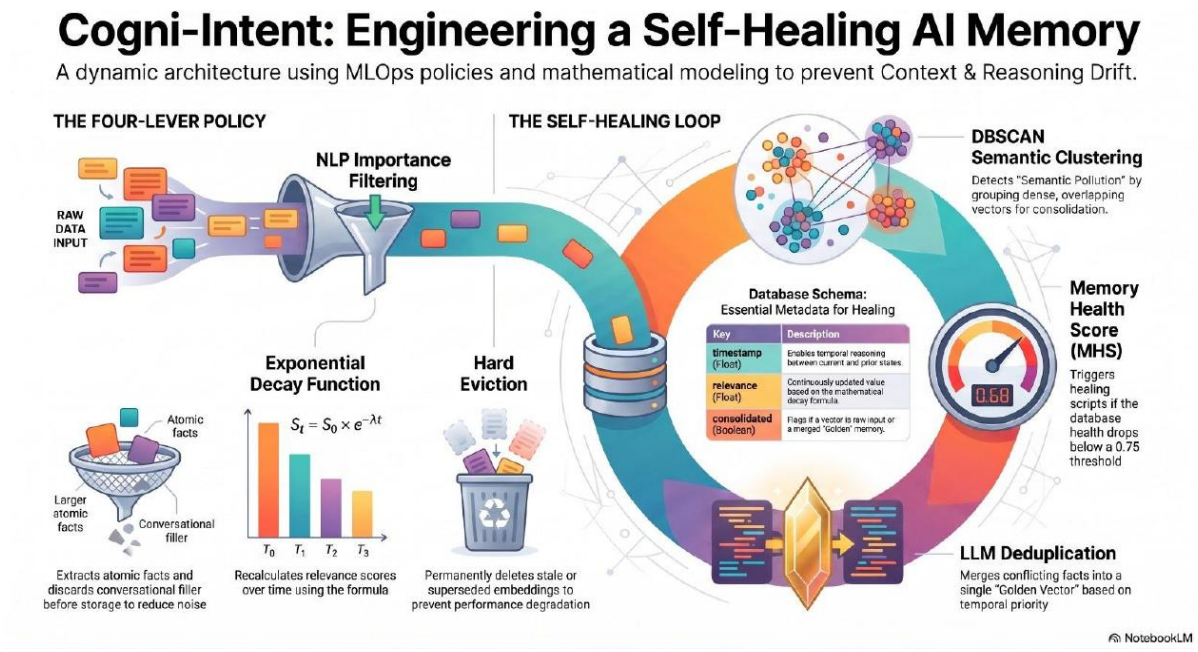


Figure 10.2: Early design: four-lever policy and self-healing loop (hard eviction later replaced by soft archiving)

Figure 10.2 shows an earlier design iteration with a hard-eviction lever. During development this lever was replaced by soft archiving once the diachronic-continuity problem was recognised. The final system therefore never deletes a clinical record.

10.3 Current limitations or known bugs

- **Synthetic scenarios.** The demonstrations use simulated clinical notes. The platform has not been validated on real patient data and is not a medical device.

- **LLM dependence.** The quality of a Golden Fact depends on the LLM; the NLI guardrail reduces but cannot completely remove the chance of a poor merge, and live consolidation needs a Gemini API key and network access.
- **Polling interval.** The daemon scans every 60 seconds, so a contradiction can exist in memory for up to a minute before it is healed.
- **Client-side engine is an approximation.** The browser engine reproduces the behaviour for demonstration but is not a substitute for the full UMAP/HDBSCAN/NLI pipeline on large stores.
- **Threshold tuning.** The 0.75 threshold, decay rate and cluster density are sensible defaults but must be tuned for each domain.
- **Scalability.** The prototype was exercised with small memory stores; behaviour at millions of vectors has not been benchmarked.
- **No end-user authentication.** Role-based access control is planned but not yet implemented.

CHAPTER 11
Future Enhancements

11.1 Planned features

- **Graph-based memory.** Store lineage as an explicit graph so that questions such as "what did the agent believe on day 20?" can be answered directly.
- **Human-in-the-loop review.** Let a clinician or administrator approve or reject a Golden Fact before it replaces the originals.
- **Role-based access and authentication.** Token-based API access with agent, auditor and administrator roles, plus per-patient scoping.
- **Adaptive thresholds.** Learn the MHS threshold and decay rate for each domain from feedback instead of fixing them by hand.
- **Multi-domain scenarios.** Extend the Simulation Lab to finance, legal and customer-support memories.
- **Multi-agent memory.** Share one healed memory between several cooperating agents with conflict resolution.

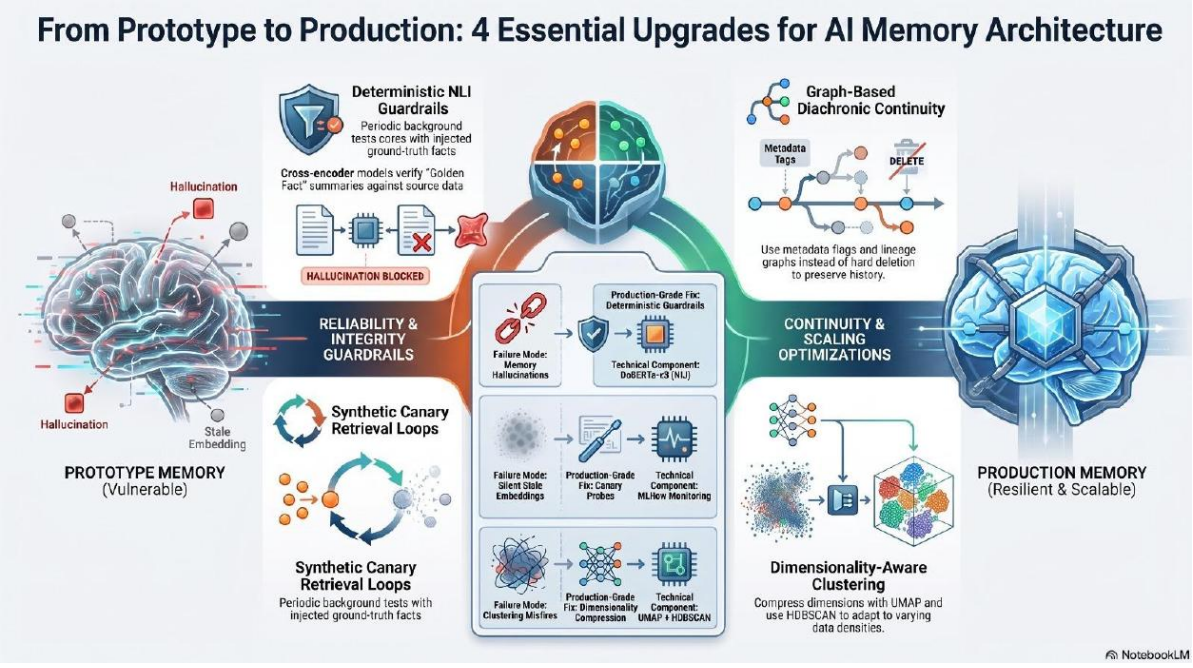


Figure 11.1: From prototype to production: four essential upgrades for AI memory architecture

11.2 Possible integrations or optimizations

- **MLflow dashboards** for continuous health, drift and canary metrics, with alerts when the loop stops or accuracy falls.

- **Qdrant or another production vector store** behind the existing abstraction layer for horizontal scaling.
- **Containerisation and CI/CD** with Docker and GitHub Actions to run the test suite on every push.
- **GPU acceleration and incremental clustering** to avoid re-clustering the whole store at each cycle.
- **Alternative LLMs and local models** so that healing can run without an external API.
- **EHR and FHIR connectors** for pilot studies in clinical decision-support systems.

CHAPTER 12
Conclusion

12.1 Summary of the project

Long-running AI agents depend on memories that grow and change over time, yet the standard vector-retrieval approach cannot recognise that a newer note may replace an older one. Cogni-Intent addresses this with an autonomous, self-healing memory layer. Notes are embedded as 384-dimensional Sentence-BERT vectors and stored in ChromaDB. A 60-second background daemon reduces the vectors with UMAP, clusters them with HDBSCAN, verifies contradictions with a cross-encoder NLI model and computes a Memory Health Score. When the score drops below 0.75, an LLM merges the conflicting notes into a Golden Fact, the NLI guardrail verifies it, and the superseded records are soft-archived with lineage links.

12.2 What was achieved

- A complete, working platform: FastAPI back end, ChromaDB store, ML pipeline, LLM healer and an interactive web front end.
- An end-to-end demonstration, through the Simulation Lab and Doctor Chat, of the difference in agent behaviour before and after healing.
- A safe design that never deletes data: every Golden Fact is traceable to the notes it replaced.
- Dual-engine deployment that runs on GitHub Pages without a server and on a local FastAPI service with live ChromaDB.
- An automated test suite covering encoding, soft eviction, drift detection and healer schemas, plus extensive documentation and a plain-English user manual.

12.3 Skills learned during development

- Vector databases, embeddings and retrieval-augmented generation, including their failure modes.
- Unsupervised learning in practice: UMAP, HDBSCAN and the effect of dimensionality on clustering.
- Natural language inference and cross-encoder models for verification of generated text.
- Prompt design and schema validation for LLM-based processing.

- Building REST APIs and background tasks with FastAPI; API testing with Postman and Swagger.
- MLOps thinking: health metrics, thresholds, canary probes, monitoring and safe rollback by archiving.
- Front-end engineering for a dual-mode (static and server-backed) application, Git/GitHub workflow and technical documentation.

In conclusion, Cogni-Intent shows that agent memory should be monitored and maintained like any other production system. By combining unsupervised drift detection, NLI verification, LLM consolidation and lineage-preserving archives, it stops memory confusion without sacrificing the history that regulated domains must keep.

CHAPTER 13
References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.

[3] W. Wang et al., "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[4] L. McInnes, J. Healy and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for dimension reduction," arXiv:1802.03426, 2018.

[5] R. J. G. B. Campello, D. Moulavi and J. Sander, "Density-based clustering based on hierarchical density estimates," in Advances in Knowledge Discovery and Data Mining (PAKDD), 2013, pp. 160-172.

[6] L. McInnes, J. Healy and S. Astels, "hdbscan: Hierarchical density based clustering," Journal of Open Source Software, vol. 2, no. 11, p. 205, 2017.

[7] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in Proc. KDD, 1996, pp. 226-231.

[8] P. He, J. Gao and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," arXiv:2111.09543, 2021.

[9] A. Williams, N. Nangia and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in Proc. NAACL-HLT, 2018, pp. 1112-1122.

[10] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824-836, 2020.

[11] Chroma, "Chroma documentation: the AI-native open-source embedding database." [Online]. Available: https://docs.trychroma.com

[12] S. Ramirez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[13] Sentence-Transformers documentation, "all-MiniLM-L6-v2 model card." [Online]. Available: https://www.sbert.net

[14] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev

[15] MLflow, "MLflow documentation." [Online]. Available: https://mlflow.org/docs/latest

[16] GitHub, "GitHub Pages documentation." [Online]. Available: https://pages.github.com

[17] Cogni-Intent project repository, README.md and architecture infographics (project documentation).

CHAPTER 14
Appendices

14.1 Screenshots of the app

The architecture infographics that document the design appear as Figures 4.2 to 4.6 in Chapter 4, Figures 10.1 and 10.2 in Chapter 10 and Figure 11.1 in Chapter 11. Screenshots of the running application (landing page, Simulation Lab before and after healing, Doctor Chat and the history tree) are to be inserted below from the project demonstration.

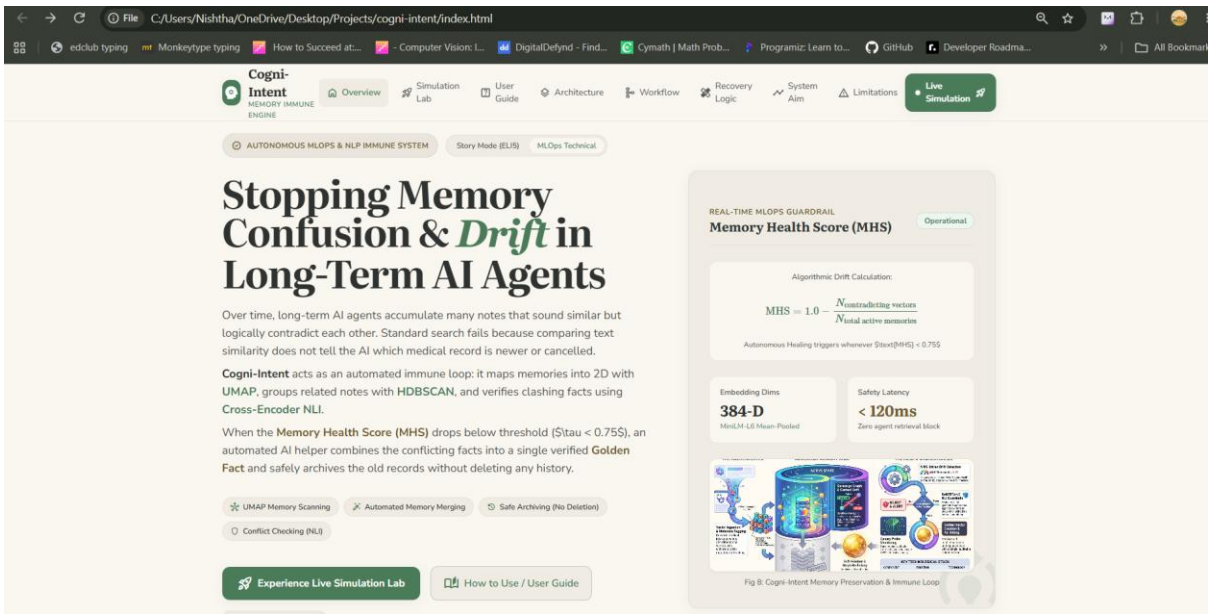


Figure 14.1: index.html: landing page

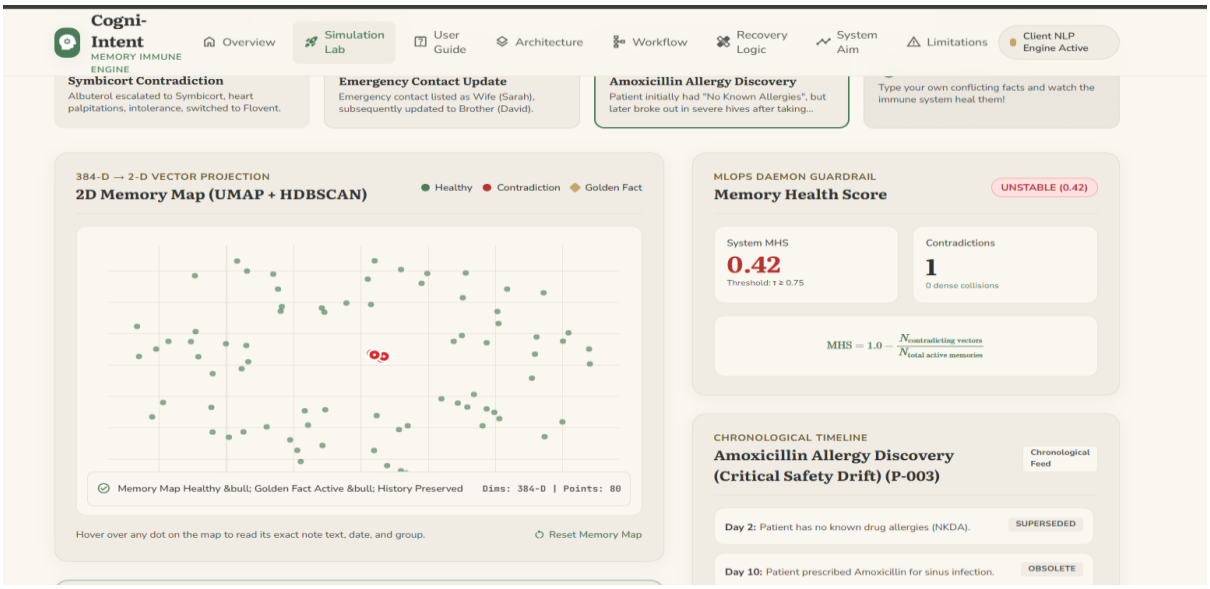


Figure 14.2: simulation.html: Simulation Lab before healing

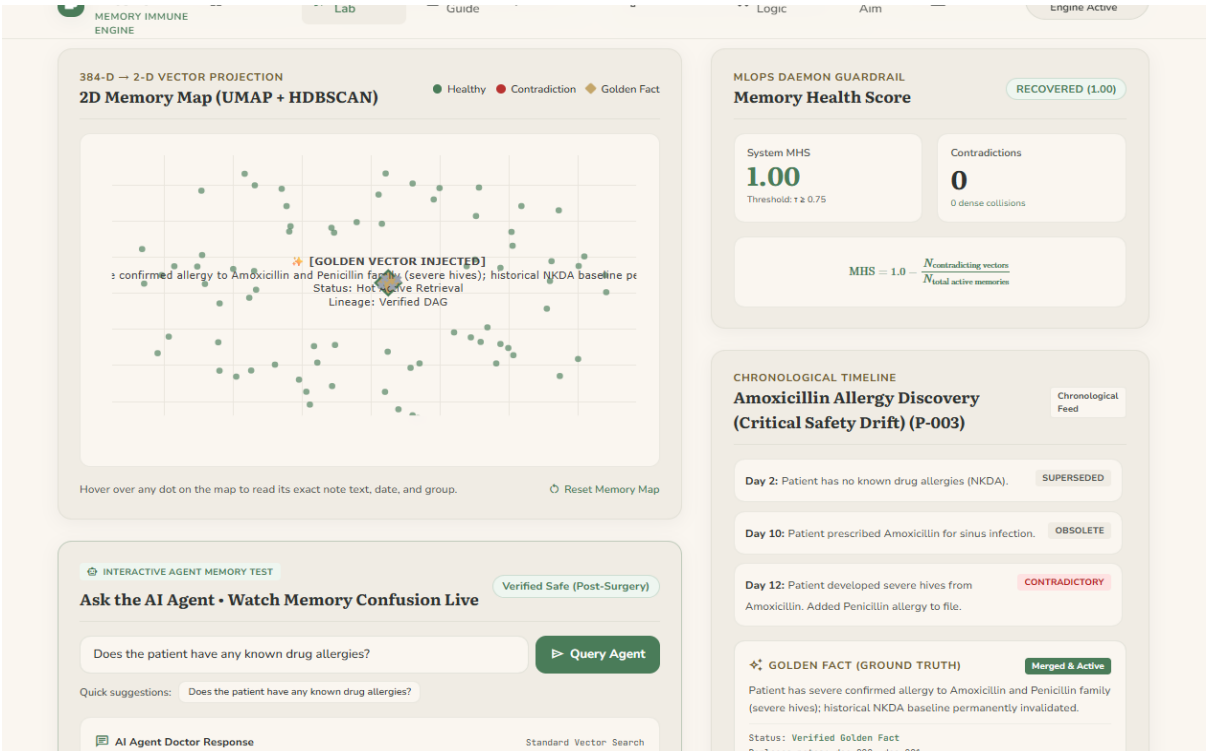


Figure 14.3: simulation.html: after healing (Golden Fact and history tree)

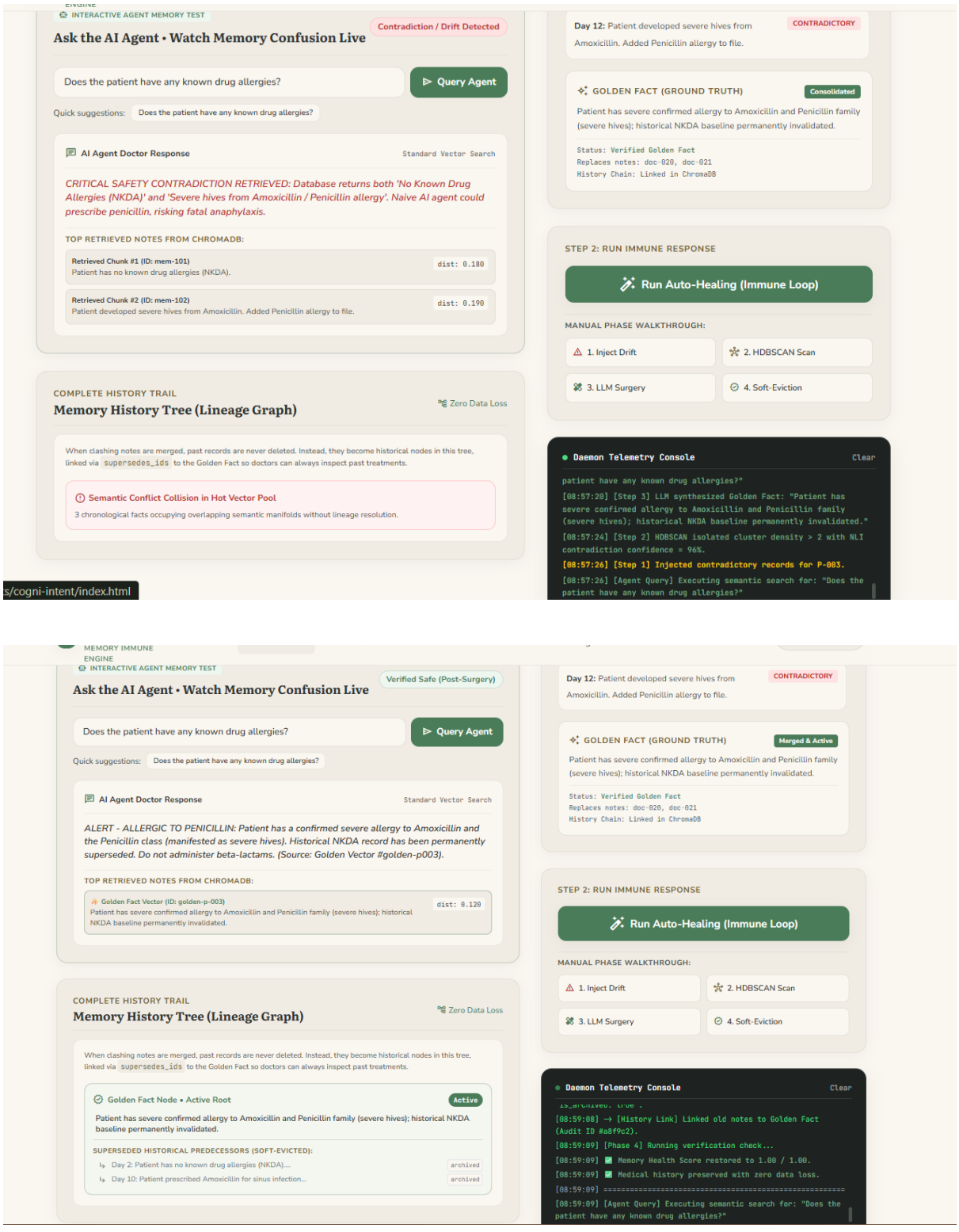


Figure 14.4: Doctor Chat sandbox: before vs. after healing

14.2 Sample code snippets

The snippets below are short, representative sketches of the core logic described in Chapter 6 (temporal decay, health gating, soft archiving and NLI verification). They illustrate the approach and are not a verbatim copy of the repository source.

Temporal decay of relevance

```
import math, time

def decay(score0: float, created_ts: float, lam: float, now: float | None = None) ->
float:
    """S_t = S_0 * exp(-lambda * t), t in days"""
    now = now or time.time()
    t_days = (now - created_ts) / 86400
    return score0 * math.exp(-lam * t_days)
```

One pass of the immune loop (simplified)

```
def immune_pass(store, nli, healer, cfg):
    recs = store.get_active()                       # is_archived == False
    X = umap.UMAP(n_components=2, metric="cosine").fit_transform(recs.embeddings)
    labels = hdbscan.HDBSCAN(min_cluster_size=2).fit_predict(X)
    mhs = health_score(recs, labels, nli, cfg)      # 0.0 - 1.0
    log_health(mhs)
    if mhs >= cfg.contradiction_threshold:          # 0.75 by default
        return mhs
    for cluster in dense_clusters(recs, labels, cfg.max_cluster_density):
        notes = sorted(cluster, key=lambda r: r.timestamp)
        golden = healer.merge(notes)                # LLM, newest facts win
        if not nli.verify(golden.text, [n.text for n in notes]):
            alert("Golden Fact rejected by NLI guardrail")
            continue
        gid = store.add(golden.text, consolidated=True,
                        supersedes_ids=[n.id for n in notes])
        for n in notes:
            store.update_metadata(n.id, is_archived=True)   # soft eviction, no delete
    return mhs
```

Drift-aware metadata record

```
{
  "id": "mem_0042",
  "document": "Escalated to Symbicort.",
  "metadata": {
    "timestamp": 1760000000.0,
    "relevance_score": 0.82,
    "is_consolidated": false,
    "is_archived": false,
    "supersedes_ids": []
  }
}
```

14.3 Installation/setup instructions

```
# 1. Clone the repository
git clone https://github.com/<your-username>/cogni-intent.git
cd cogni-intent


# 2. Install dependencies (virtual environment recommended)
pip install -r requirements.txt
```

```
# 3. (Optional) Gemini key for live LLM consolidation
export GEMINI_API_KEY="your_api_key_here"      # Linux/macOS
$env:GEMINI_API_KEY="your_api_key_here"         # Windows PowerShell

# 4. Start the FastAPI MLOps daemon
python src/api/main.py

# 5. Run the automated tests
python tests/run_tests.py
```

- Interactive web UI: http://localhost:8080
- Swagger documentation: http://localhost:8080/docs
- Health endpoint: http://localhost:8080/api/health

14.4 User manual or guide

The full plain-English guide is available as manual.html in the repository. Its main parts are:

- **60-second crash course:** the problem, the solution and the result in simple terms.
- **5-step Simulation Lab walkthrough:** choose a clinical scenario; test the Doctor Chat before healing; run the auto-healing immune loop; test the Doctor Chat again; inspect the healed Golden Fact and archived history tree.
- **Website navigation guide:** a tour of index, simulation, architecture, workflow, recovery, lifecycle and limitations pages.
- **FAQ:** client-side vs. back-end execution, the Memory Health Score, and how to inject a custom scenario.

14.5 Certifications

Certificates of any courses, workshops or competitions related to this project (for example in machine learning, vector databases or MLOps) may be attached on this page.